

■ ■ SETTING THE FRAME

Who here has tried agentic search?

Hands up.

Two products, two retrieval problems

A stock library and a DAM system look similar from the outside. The search problem underneath is not the same one.



Stock photo library

One shared catalogue, many unrelated buyers. Every user searches the same corpus with no prior relationship to any of it. Intent is broad and visual — "two people laughing in an office, lots of copy space".

Millions of assets, one index, and no boundary between users — everybody searches the same catalogue.



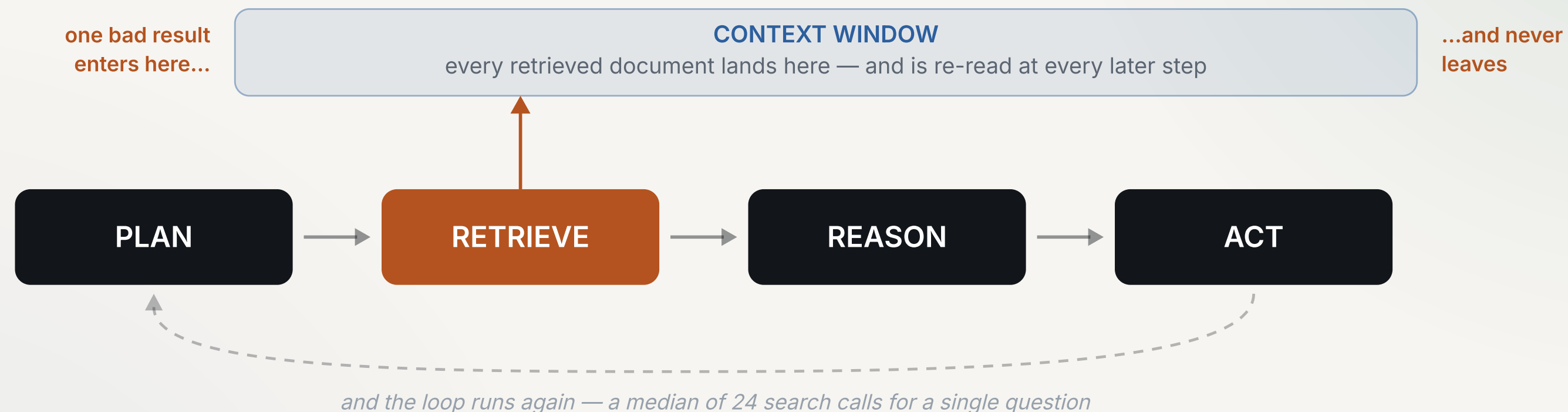
Digital Asset Management (DAM) system

One customer's own assets, and they know what's in there. The user is often looking for a specific known item — a campaign shoot, a signed contract, last year's product video.

Every query is scoped to one tenant before it ranks — the boundary is part of the retrieval problem, not a detail around it.

Your agents are only as good as your **retrieval**

Agentic search is a loop — plan, retrieve, reason, act — run many times over one question. **Retrieve is the only step that gates what the model ever gets to see.**



A bad result doesn't visit. It moves in.

A false positive is not a wasted slot on a page of results that the user ignores. It enters the context, gets **re-read at every later step**, and shapes the next action. Retrieval error does not just persist — it compounds.

Which is why the rest of this is retrieval

Not because the agent part is uninteresting, but because it is the part we can least afford to get wrong.

dense embeddings

exact match

late interaction

the funnel

the vector store

our own eval set

Tied on quality, 1,431 $\times$ apart on cost

Agents don't search once. When a workflow fans out **fifty queries where a person issued one**, cost per query stops being a rounding error and becomes the architecture.

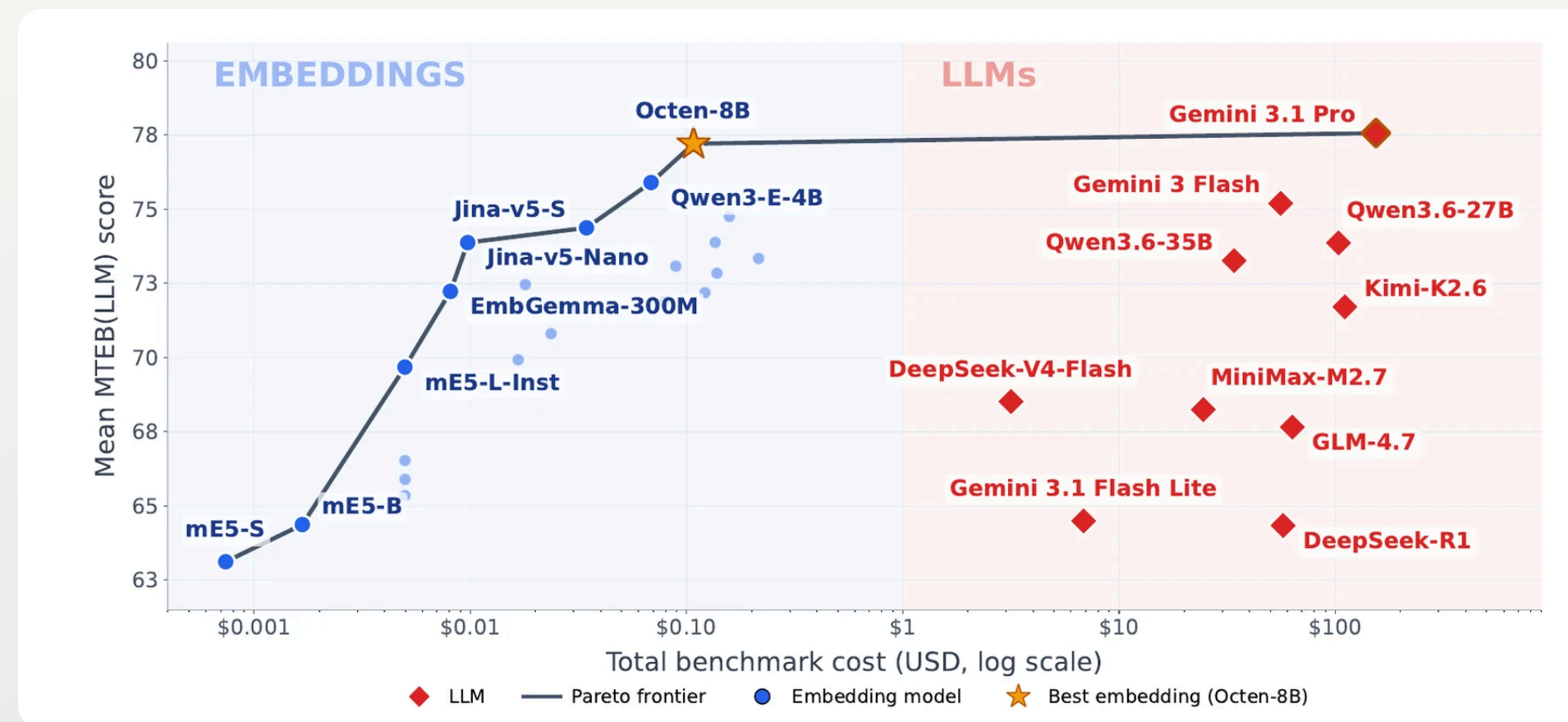


Figure 1 from El Assadi, Muennighoff & Lee, *The Embedder's Dilemma* · COLM 2026. Note the log scale on cost.

0.4
points apart

Best LLM (Gemini 3.1 Pro, 77.6) against best embedding model (77.2), over 37 tasks. In aggregate, tied.

1,431 $\times$
the cost of closing it

USD 154 against USD 0.11 per benchmark pass — and 2.5–736 $\times$ slower on the same GPU.

Late interaction sits in the middle

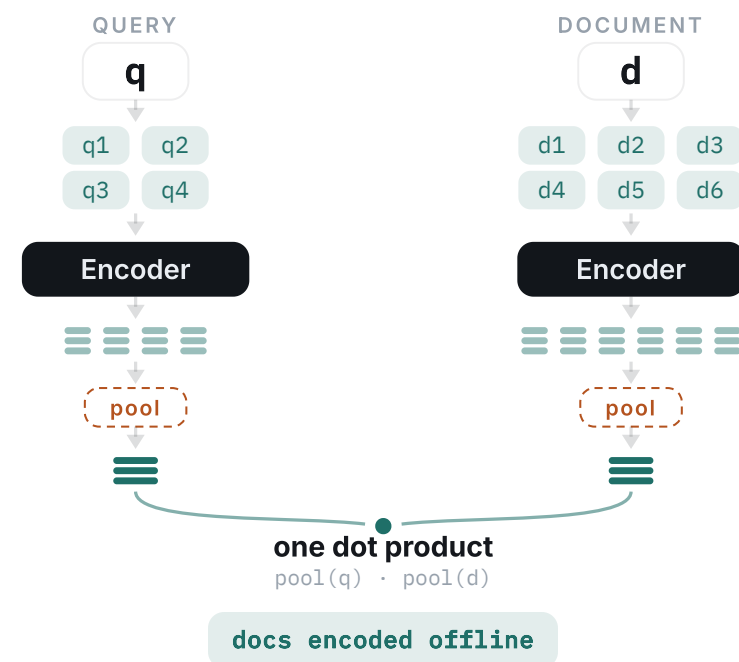
Three ways to score a query against a document. The difference is **when the two sides are allowed to meet**.

← cheaper · more scalable

more accurate · expensive →

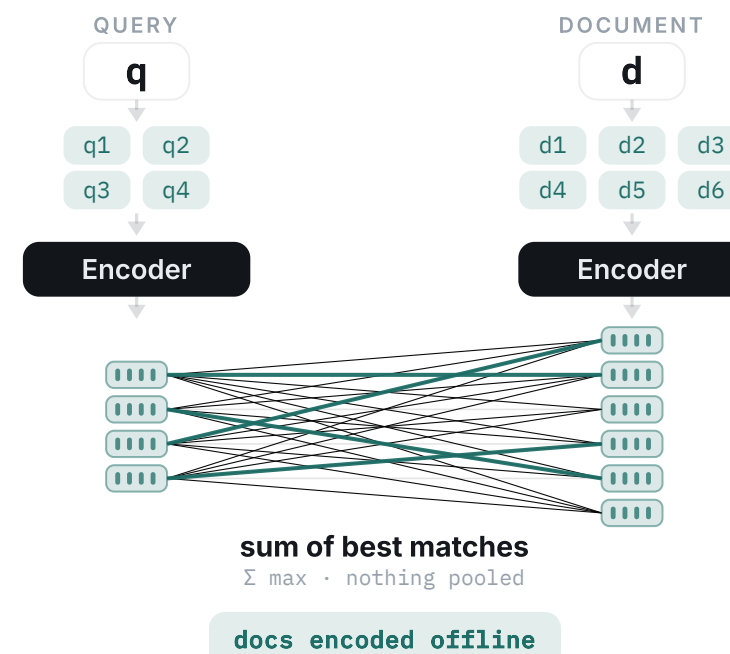
Single-vector retrieval

pooled dual encoder



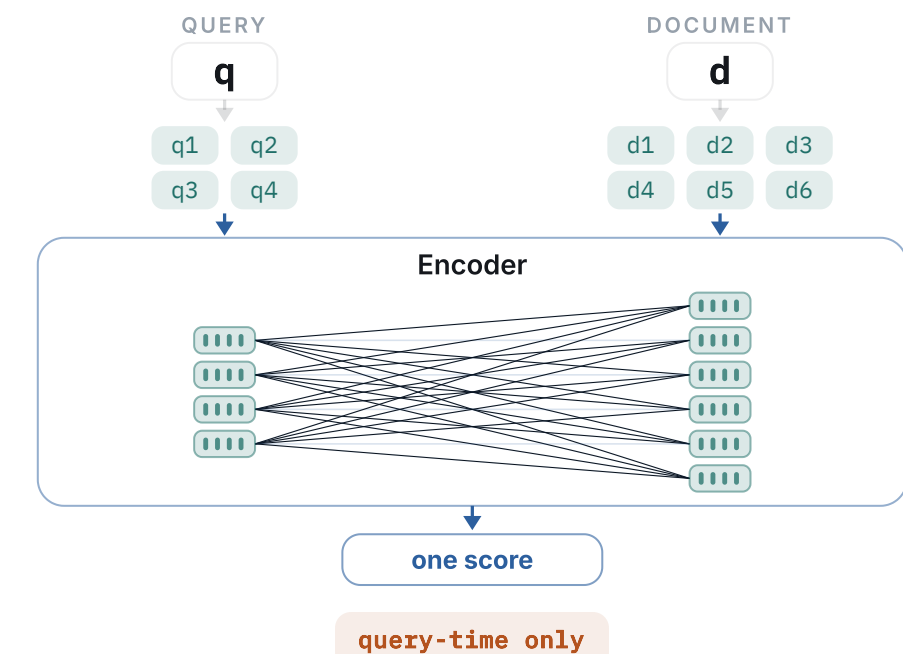
Late-interaction retrieval

multi-vector dual encoder



Full-interaction reranking

cross-encoder



That interaction has a name: **MaxSim**

For every **query token**, keep only its best-matching unit of the asset, then sum those maxima. Nothing is pooled, so a rare local clue survives.

And for us the unit is a **patch**

Same operator, patches instead of tokens — which is where the next few slides go.

An agent searching a DAM needs **all of it**

Not four separate projects. Four preconditions for the same thing — and a DAM is where they all bind at once: a customer's own assets, in every format, where a near miss is a failure.

It has to see

Content understanding over images, video, PDF pages and the long tail — because the evidence is in the asset, not the caption someone forgot to write.

PATCHES, NOT METADATA

It has to be right

Late interaction where precision decides the outcome, and an eval set built from our own customers' queries rather than a public leaderboard.

NEAR MISS = FAILURE

It has to afford to ask

An agent fans out. The funnel, the compression and the embedding-first economics are what make the fiftieth query as cheap as the first.

COST IS THE DESIGN

It has to stay ours

Self-hosted on EU infrastructure, so we can swap the model, re-index on our own terms, and never move a customer's assets out of jurisdiction.

SOVEREIGN BY ARCHITECTURE

Which leaves the interesting questions open

What does an agent need from search that a person never asked for — an explanation, a confidence, a second opinion? What does our eval set become when the caller is a machine that will happily ask a hundred times? And which of these do we build next?